

יחידה 4.1 – להכניס בינה לקוד (API)

מהו API? פונקציית עטיפה שמחזירה תשובה



יחידה 4.1 – להכניס בינה לקוד (API)

4 דקות

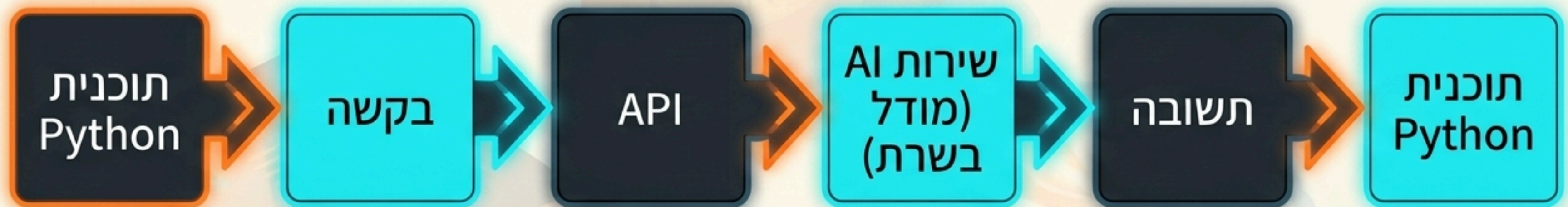
פתיחה: איך תוכנית "מדברת" עם מודל?

אפליקציה "חכמה" עונה על שאלות
בעברית תוך שנייה.

איך תוכנית פיתון "מדברת" עם מודל
שיושב על שרת של מישהו אחר?

יחידה 4.1 – להכניס בינה לקוד (API)

פתיחה: התשובה



יחידה 4.1 – להכניס בינה ללקוד (API)

משל המלצר

ה-API הוא המלצר.

התוכנית היא הלקוח.

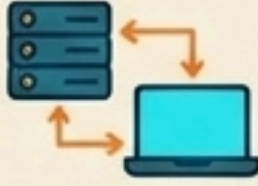
השרת והמודל הם המטבח.

התשובה היא המנה.

התוכנית לא מכילה את כל המודל — היא רק שולחת בקשה ומקבלת תשובה, כמו שהלקוח לא נכנס למטבח בעצמו.

יחידה 4.1 – להכניס בינה לקוד (API)

מושגי יסוד

הערוץ שדרכו תוכנית שולחת בקשה ומקבלת תשובה משירות חיצוני	API 
מזהה ומאשר את הפרויקט השולח — לא סיסמה להקליד	מפתח API 
האובייקט בקוד שמנהל את החיבור לשירות	לקוח (client) 
הטקסט שנשלח למודל כבקשה	פרומפט 
מה שהמודל מחזיר	תשובה (response) 
מספר הקריאות המותר בפרק זמן נתון	הגבלת קצב 
פעולה שמסתירה את הקריאה לשירות מאחורי קריאה פשוטה	פונקציית עטיפה 

יחידה 4.1 – להכניס בינה לקוד (API)



8 דקות

מוצאים את התקלה

```
client = genai.Client(api_key="my-real-secret-key")
```

**מה הבעיה כאן, ומה קורה אם הקובץ הזה יעלה למאגר
משותף או ישותף במקרה?**

יחידה 4.1 – להכניס בינה לקוד (API)

מוצאים את התקלה: התשובה

```
client = genai.Client(api_key="my-real-secret-key")
```

המפתח כתוב גלוי בתוך הקוד – כל מי שרואה את הקובץ יכול להשתמש בו.
תיקון: `genai.Client()` בלי מפתח גלוי, קורא את `GEMINI_API_KEY` ממשתנה סביבה.

מהדגמה החיה לפונקציית עטיפה

```
def ask_ai(prompt):  
    response = client.models.generate_content(model=MODEL, contents=prompt)  
    return response.text
```

קריאה בודדת **חיה** מוצגת קודם, ואז אותה קריאה עוברת לתוך פעולה —
return, לא print, כדי שהקוד הקורא יקבל את התשובה בחזרה.

למה return, לא print



אובייקט מוחשי לשימוש עתידי



תוצאה חולפת, נעלמת מיד

ערך שחוזר עם return אפשר לשמור, להשוות או לשלב בטקסט נוסף. ערך שרק print מציג נעלם ברגע שהוא מופיע על המסך. תשובת המודל היא ניחוש שמבוסס הסתברות – בודקים אותה, כמו כל קלט אחר.

מצב כיתה: המעבדה

**הקבצים במעבדה פועלים במצב כיתה –
ask_ai() מחזירה משפט קבוע, בלי חיבור
אמיתי לשירות. זה מכוון, לא קיצור דרך.**

יחידה 4.1 – להכניס בינה לקוד (API)



חימום: עטיפה עם שתי תקלות

```
def ask_ai():  
    response = "Mock response: balance study, movement, rest, and sleep."  
    print(response.text)  
  
answer = ask_ai("Give one healthy study habit.")  
print(answer)
```

הריצו וקראו את השגיאה.

חימום: שתי התקלות, בסדר

```
Traceback (most recent call last):  
  File "msvtrompt.py", line 18, in <module>  
    prompt = prompt  
  File "C:\Users\wittms-Devklopment.py", line 38, in <module>  
    return ai(ask_ai())  
TypeError: ask_ai() takes 0 positional arguments but 1 was given
```

תקלה 1: prompt חסר כפרמטר.
אחרי התיקון:

```
Traceback (most recent call last):  
  File "msrtin.py", line 31, in <module>  
    nent = acnk()  
  File "C:\Users\wdithus-Development.py", line 37, in <module>  
    return ask_ai(-prompt, 'text')  
AttributeError: 'str' object has no attribute 'text'
```

תקלה 2: השורה המדומה כבר
מחזירה טקסט רגיל – text. לא
מתאים לה, ויילא הפעולה צריכה
return, לא print.

משימה 1: הפרמטר return-וה, מאושרים

8 דקות

אחרי התיקון,

```
answer = ask_ai("Give one healthy study habit.");  
print(answer)
```

בדקו, **והסבירו במשפט אחד** למה `answer` היה מקבל `None`
אם `ask_ai` הייתה עדיין משתמשת ב-`print`.

דקות 12

משימה 2: בדיקת הפרומפט לפני "הקריאה"

```
The prompt is too short. Please add details.  
Mock response: balance study, movement, rest, and sleep.
```

אם `len(prompt.strip()) < 5`,
מחזירים "

**"The prompt is too short.
Please add details."**

בלי להמשיך הלאה;
אחרת מחזירים את התשובה.

בדקו עם "Hi" ועם פרומפט
רגיל.



דיון: כמה קריאות, ולמה זה משנה

```
count = 0
while count < 20:
    print(ask_ai("Give one study tip."))
    count += 1
```



כמה קריאות נשלחות כאן? למה זה עלול להיות
בעייתי גם עם פונקציה מדומה, ובעייתי באמת עם
עם שירות אמיתי?

יחידה 4.1 – להכניס בינה לקוד (API)

דיון: התשובה



20 קריאות. גם פונקציה מדומה מבזבזת זמן ריצה; שירות אמיתי מוסיף זמן, מכסה, הגבלת קצב ואולי עלות.
פתרון (יחידה 4.2): לאסוף הכול קודם, ולשלוח קריאה אחת אחת בלבד.



שומרים + בדיקת יציאה

שומרים: `G8_U4_M1_Wrapper_<Name>.py`

1. API הוא...
2. מפתח API נשמר ב... ולא ב...
3. בפונקציית עטיפה משתמשים ב-`return` משום ש...

בדיקת יציאה: תשובות לדוגמה

1. הערוץ שדרכו תוכנית שולחת בקשה ומקבלת תשובה
(**AP הוא...**) משירות חיצוני.

2. במשתנה סביבה – ולא בקובץ, בשיחה או בצילום מסך.
(**מפתח API נשמר ב... ולא ב...**)

3. ...כדי שהקוד שקורא לפעולה יקבל את הערך בחזרה ויוכל
לשמור, להשוות או להשתמש בו הלאה.
(**בפונקציית עטיפה משתמשים ב-return משום ש...**)